

Day 2 — Knowledge Check Quiz

15 questions covering Delta Lake Advanced, Optimization & Data Management, Cost Management, and Security & Governance.

Section 1: Delta Lake Advanced (Q1–Q3)

Q1. Change Data Feed

What does the `_change_type` column contain when reading Change Data Feed (CDF)?

- A) The name of the user who made the change
- B) The SQL statement that caused the change
- C) One of: `insert`, `update_preimage`, `update_postimage`, `delete`
- D) The timestamp of the change only

Answer

****C)** One of: `'insert'`, `'update_preimage'`, `'update_postimage'`, `'delete'` CDF tracks the type of change at the row level. `'update_preimage'` is the row BEFORE the update, `'update_postimage'` is the row AFTER.

Q2. Enabling CDF

Which command enables Change Data Feed on an existing Delta table?

- A) `ALTER TABLE my_table ENABLE CDF`
- B) `ALTER TABLE my_table SET TBLPROPERTIES (delta.enableChangeDataFeed = true)`
- C) `CREATE TABLE my_table WITH (cdf = true)`
- D) CDF is enabled by default on all Delta tables

Answer

****B)** `ALTER TABLE my_table SET TBLPROPERTIES (delta.enableChangeDataFeed = true)` CDF must be explicitly enabled per table via a table property.

Q3. Reading CDF Data

Which function reads change data from a CDF-enabled table starting from version 5?

- A) `SELECT * FROM my_table VERSION AS OF 5`
- B) `DESCRIBE HISTORY my_table`
- C) `SELECT * FROM my_table WHERE _commit_version >= 5`
- D) `SELECT * FROM table_changes('my_table', 5)`

Answer

****D)** `'SELECT * FROM table_changes('my_table', 5)'` `table_changes()` is the SQL function to read CDF data. `'VERSION AS OF'` reads a snapshot, not changes. `'DESCRIBE HISTORY'` shows metadata only.

Section 2: Optimization & Data Management (Q4–Q9)

Q4. Small Files Problem

What is the main consequence of having thousands of small files in a Delta table?

- A) Data corruption risk
- B) Schema enforcement failures
- C) Slow query performance due to excessive file-open overhead
- D) Loss of ACID guarantees

Answer

****C)** Slow query performance due to excessive file-open overhead** Each file requires an open/read/close operation. Thousands of small files lead to high I/O overhead and slow queries. 'OPTIMIZE' compacts them.

Q5. OPTIMIZE

What does `OPTIMIZE my_table` do?

- A) Compacts small files into larger, optimally-sized files
- B) Deletes old data versions
- C) Reorders data by primary key
- D) Rebuilds the Delta transaction log

Answer

****A)** Compacts small files into larger, optimally-sized files** OPTIMIZE merges many small Parquet files into fewer, larger files (target ~1GB) without changing data content.

Q6. Z-ORDER vs Liquid Clustering

Which statement about Liquid Clustering is correct?

- A) Liquid Clustering requires running `OPTIMIZE ZORDER BY` jobs
- B) Liquid Clustering is only available for external tables
- C) Liquid Clustering automatically reorganizes data incrementally — no scheduled OPTIMIZE needed
- D) Liquid Clustering and Z-ORDER are the same feature

Answer

****C)** Liquid Clustering automatically reorganizes data incrementally — no scheduled OPTIMIZE needed** Liquid Clustering is the modern replacement for partitioning + Z-ORDER. It reorganizes data incrementally and automatically during writes, removing the need for manual OPTIMIZE ZORDER jobs.

Q7. VACUUM Safety

What happens if you run `VACUUM my_table RETAIN 0 HOURS` ?

- A) Nothing — VACUUM has a safety check preventing this
- B) All historical file versions are deleted, breaking time travel
- C) Only the current version is kept, but time travel still works
- D) The table is dropped

Answer

****A) Nothing — VACUUM has a safety check preventing this**** By default, Delta Lake prevents VACUUM with retention less than 7 days (168 hours). You must explicitly set ``spark.databricks.delta.retentionDurationCheck.enabled = false`` to override this safety check.

Q8. Partitioning Best Practices

When should you avoid partitioning a Delta table?

- A) When the partition column has very few distinct values
- B) When using a SQL Warehouse for queries
- C) When the table is very large (>1TB)
- D) When each partition would contain less than 1 GB of data

Answer

****D) When each partition would contain less than 1 GB of data**** Partitioning by high-cardinality columns (e.g., `customer_id`) creates thousands of tiny partitions, worsening the small files problem. Each partition should have ≥ 1 GB of data.

Q9. Deletion Vectors

What are Deletion Vectors in Delta Lake?

- A) A way to track deleted rows without rewriting data files
- B) A type of index for faster lookups
- C) A backup mechanism for disaster recovery
- D) A compression algorithm for Parquet files

Answer

****A) A way to track deleted rows without rewriting data files**** Deletion Vectors mark rows as deleted in a small side file instead of rewriting the entire Parquet file. This makes DELETE/UPDATE/MERGE operations much faster.

Section 3: Cost Management (Q10–Q12)**Q10. Compute Cost Drivers**

Which compute type is the most cost-efficient for running production ETL pipelines?

- A) Job Cluster (auto-terminates after job)
- B) All-Purpose Cluster (always running)
- C) SQL Warehouse (serverless)
- D) GPU Cluster

Answer

****A) Job Cluster (auto-terminates after job)**** Job Clusters are ~2x cheaper per DBU than All-Purpose Clusters and automatically terminate after the job completes. They are the recommended choice for production pipelines.

Q11. Incremental vs Full Load Cost

Why is incremental processing (MERGE/Auto Loader) cheaper than full reload?

- A) It uses less storage only
- B) It runs on serverless compute automatically
- C) It avoids Delta Lake fees

- D) It processes only new/changed data, consuming fewer compute resources

Answer

****D)** It processes only new/changed data, consuming fewer compute resources** Incremental processing (MERGE, Auto Loader with checkpoint) scans and processes only new or changed records, dramatically reducing compute time and DBU costs.

Q12. OPTIMIZE Cost Trade-off

Running `OPTIMIZE` costs compute DBUs. When is this cost justified?

- A) Never — OPTIMIZE is free
- B) When the table has many small files and is queried frequently
- C) Only for Bronze tables
- D) Only before running VACUUM

Answer

****B)** When the table has many small files and is queried frequently** OPTIMIZE costs compute time to compact files, but each subsequent query reads fewer files, saving much more compute over time. The ROI is highest for frequently-queried tables with many small files.

Section 4: Security & Governance (Q13–Q15)

Q13. Unity Catalog Privileges

A data engineer runs `GRANT SELECT ON TABLE orders TO analysts`. What access do analysts get?

- A) Full read and write access to the orders table
- B) Access to all tables in the schema
- C) Read-only access to the orders table
- D) Admin access to the catalog

Answer

****C)** Read-only access to the orders table** `GRANT SELECT` provides read-only access to the specific table. `MODIFY` would be needed for write access. Privileges do not cascade to other tables.

Q14. Row-Level Security

How do you implement row-level security in Unity Catalog?

- A) Create dynamic views with `current_user()` or `is_member()` in the WHERE clause
- B) Use `ALTER TABLE ADD CONSTRAINT` with user filters
- C) Set table properties with `row.filter.enabled = true`
- D) Define row filters in the cluster configuration

Answer

****A)** Create dynamic views with `current_user()` or `is_member()` in the WHERE clause** Row-level security is implemented through dynamic views that filter rows based on the current user's identity or group membership using session functions.

Q15. Column Masking

Which approach masks email addresses so only members of the `pii_readers` group see the real value?

- A) `ALTER TABLE customers DROP COLUMN email`
- B) Create a SQL function that checks `is_member('pii_readers')` and apply it with `ALTER TABLE ... SET MASK`
- C) Use `DENY SELECT` on the email column
- D) Encrypt the column with a cluster-level key

Answer

****B)** Create a SQL function that checks `is_member('pii_readers')` and apply it with `ALTER TABLE ... SET MASK`** Column masking uses SQL functions that conditionally reveal or mask data based on the user's group membership. The mask function is applied to the column via `ALTER TABLE ALTER COLUMN SET MASK`.

Answer Key

Q	Answer	Topic
1	C	CDF _change_type
2	B	Enable CDF
3	D	table_changes()
4	C	Small files problem
5	A	OPTIMIZE compaction
6	C	Liquid Clustering
7	A	VACUUM safety check
8	D	Partitioning guidelines
9	A	Deletion Vectors
10	A	Job Cluster cost
11	D	Incremental processing
12	B	OPTIMIZE ROI
13	C	GRANT SELECT
14	A	Row-level security
15	B	Column masking